Stack Storm

System Design Specification (SDS)

By

Amarawickrama R.H.
SE/2021/004
S. Thanoogithan
SE/2021/005
B.K. Pabasara
SE/2021/009
Lakshani A.T.
SE/2021/041

A report submitted in partial fulfilment of the requirements for the degree of Bachelor of Science Honors in Software Engineering (B.Sc.SE)

Software Engineering Teaching Unit
Faculty of Science
University of Kelaniya Sri Lanka
2026

Version History

Version	Date	Author(s)	Summary of Changes
01	2026/04/18	Amarawickrama R.H. S. Thanoogithan B.K. Pabasara Lakshani A.T.	Initial SDS draft
02	2026/10/25	Amarawickrama R.H. S. Thanoogithan B.K. Pabasara Lakshani A.T.	Updated SDS draft

List of Figures

Figure No.	Caption	Page
Figure 1	High-Level Layered Architecture Diagram	02
Figure 2	Cloud-Native Architecture Diagram	03
Figure 3	Entity-Relationship (ER) Diagram	06
Figure 4	Class Diagram – Core Domain Model	08
Figure 5	Sequence Diagram – Customer Registration	10
Figure 6	Sequence Diagram – Product Search and Filtering	11
Figure 7	Sequence Diagram – Add to Cart and Checkout	11
Figure 8	Sequence Diagram – Order Tracking	12
Figure 9	Sequence Diagram – Process Return and Refund	12
Figure 10	Sequence Diagram – Manage Inventory	13
Figure 11	User Flow / Navigation Diagram	14
Figure 12	Security Architecture Diagram	17
Figure 13	Deployment Architecture Diagram	22
Figure 14	CI/CD Pipeline Overview Diagram	23

List of Tables

Table No.	Caption	Page
Table 1	Technology Stack Summary	02
Table 2	Database Normalization Summary	06
Table 3	Class Attributes and Methods Summary	09
Table 4	UI Accessibility Compliance Checklist	15
Table 5	Authentication and Authorization Levels	18
Table 6	Encryption Standards Summary	19
Table 7	Environment Specification	22
Table 8	Hardware Requirements	23

Table of Contents

1. Architectural Design

1.1 High-Level Architecture…….…….01

1.2 Cloud-Native Architecture……..……..03

1.3 Justification for Architecture Choice…….……..04

1.4 Scalability and Maintainability Considerations……...……..05

2. Database Design.

2.1 Entity-Relationship Diagram…….…….…..06

2.2 Normalization Explanation…….…….…….…06

3. Class Diagram

3.1 Core Domain Classes…….……..……08

3.2 Class Relationships and Multiplicity…….……...……..…..08

4. Sequence Diagrams

4.1 Customer Registration……...……..…...10

4.2 Product Search and Filtering…….……..…10

4.3 Add to Cart and Checkout…….……..…11

4.4 Order Tracking…….…….…….12

4.5 Process Return and Refund…….…..…..12

4.6 Manage Inventory…….……..…13

5. UI/UX Design

5.1 Wireframes and Screen Mock-Ups…….……...14

5.2 User Flow and Navigation Diagrams…….……14

5.3 Accessibility Considerations…….…….14

6. Security Design

6.1 Authentication Mechanism……...…….……16

6.2 Authorization Levels…….....…….17

6.3 Data Protection Approach…….....…….…18

6.4 Input Validation and Sanitization Strategy…….....…19

6.5 Encryption Standards…….....…19

7. Deployment Design

7.1 Hosting Plan…….…..….21

7.2 Hardware Requirements…….……22

7.3 Environment Specification……..…23

7.4 CI/CD Pipeline Overview…….……..23

8. References…….……25

CHAPTER 1: ARCHITECTURAL DESIGN

This chapter describes the overall system architecture adopted for the Dickwella Fashion Corner platform, the justification for the architectural decisions made, and the measures taken to ensure scalability and maintainability.

1.1 High-Level Architecture

A **Layered (N-Tier) Architecture combined with the MVC pattern** has been adopted for the Dickwella Fashion Corner system. The architecture is organized into the following four tiers:

Presentation Layer (View): The presentation layer is responsible for rendering the user interface in the client's web browser. It is implemented using React.js, a component-based JavaScript library, and communicates with the backend exclusively through RESTful API calls. This layer handles client-side routing, form validation, state management, and responsive rendering across devices ranging from 320px to 2560px screen widths.

Application Layer (Controller): The application layer serves as the intermediary between the presentation layer and the business logic layer. It is implemented using Node.js with the Express.js framework. This layer receives HTTPS requests, performs request validation, invokes the appropriate business logic services, and returns JSON responses. RESTful API endpoints are organised by domain (e.g., /api/products, /api/orders, /api/customers).

Business Logic Layer (Model/Service): The business logic layer encapsulates the core domain rules and processing logic of the system. It contains service classes responsible for operations such as order processing, inventory management, payment gateway integration, loyalty point calculations, RFM analysis, notification dispatching, and AI chatbot intent resolution. This layer ensures that business rules (e.g., 30-day return window, promotional code validation, loyalty tier thresholds) are enforced consistently.

Data Access Layer: The data access layer manages all interactions with the database. It is implemented using Sequelize ORM for object-relational mapping, providing an abstraction over raw SQL queries. This layer handles CRUD operations, complex queries (e.g., sales aggregation, inventory analytics), database transactions, and connection pooling.

https://drive.google.com/file/d/1TJSgPeyk_zK53zUR2fm-b1nrjpY_4kVz/view?usp=drive_link ***Figure 1: High-Level Layered Architecture Diagram***

Table 1: Technology Stack Summary

Layer	Technology	Purpose
Frontend	React.js 18+	Component-based SPA for customer and admin UIs
State Management	Redux Toolkit	Centralized application state management
Backend Framework	Node.js 20 LTS + Express.js 4	RESTful API server and request handling
Real-time	Socket.IO	WebSocket-based real-time communication
ORM	Sequelize 6	Object-relational mapping for database operations
Database	PostgreSQL 16	Primary relational data store
Cache	Redis 7	Session storage, query caching, rate limiting
Authentication	JWT + bycrypt	Token-based auth and password hashing
Payment Gateway	Stripe SDK / PayPal SDK	Payment processing integration
Email Service	SendGrid API	Transactional and marketing email delivery
File Storage	AWS S3	Product images, invoices, and bulk upload storage
Search	Elasticsearch (optional)	Full-text search with fuzzy matching
Containerization	Docker + Docker Compose	Application containerization and orchestration

1.2 Cloud-Native Architecture

The Dickwella Fashion Corner system is designed to be deployed as a cloud-native application leveraging containerization and managed cloud services.

https://drive.google.com/file/d/1FqqTTu9a7rv5WB7SCxeQKNVv_5S-jPgY/view?usp=drive_link

Figure 2: Cloud-Native Architecture Diagram

1. Client Devices Layer

This is the entry point of the system, representing the User Interface (UI) tier. Components include tablet, mobile, and desktop browsers. This layer handles the rendering of the frontend application and initiates requests to both the CDN (for static content) and the Load Balancer (for dynamic API data).

2. CDN & Content Delivery Layer

This layer optimizes the delivery of assets to reduce latency and server load. The technology employed is AWS CloudFront, which caches and serves static assets (HTML, CSS, JavaScript) and media files (images). By using edge locations closer to the user, it significantly improves page load speeds.

3. Traffic Management Layer

This layer acts as the gatekeeper for all incoming dynamic traffic using AWS ALB (Application Load Balancer). It distributes incoming application traffic across multiple targets (containers) in the Application Cluster and ensures high availability by performing health checks and routing traffic only to healthy instances.

4. Application Cluster (Compute Layer)

This is the core logic layer of the system, where business rules are processed. It is containerized for scalability. Multiple Node.js + Express containers handle RESTful API requests, authentication, and business logic. A dedicated WebSocket Container (Socket.IO) manages persistent, two-way communication channels, essential for live notifications, chat, and real-time updates.

5. Data Services Layer

This layer provides persistent and temporary storage solutions. Amazon RDS (PostgreSQL) is used for structured, relational data and complex queries. Amazon ElastiCache (Redis) reduces database load and improves performance through session caching. Amazon S3 provides secure, durable storage of user-uploaded files, documents, and large backups.

6. External Integrations Layer

This layer extends the system's functionality through third-party services, including Stripe and PayPal APIs for payment processing, SendGrid API for automated email delivery, Shipping Carrier APIs for real-time tracking, and Google Analytics for monitoring user behavior.

1.3 Justification for Architecture Choice

The Layered (N-Tier) Architecture with MVC pattern was selected for the following reasons:

- **Separation of Concerns:** Each layer has a distinct responsibility, reducing coupling between the user interface, business logic, and data persistence. This separation enables independent development and testing of each layer.
- **Team Familiarity:** The development team possesses foundational knowledge of the MVC pattern from coursework. This familiarity reduces the learning curve and accelerates development within the 6–8 month project timeline.
- **Modularity:** The service-based organization within the business logic layer allows individual modules to be developed, tested, and updated independently, aligning with the phased delivery approach outlined in the SRS.
- **Scalability Path:** While a full microservices architecture was considered, it was deemed premature for the current project scope. The layered architecture provides a clear migration path for future iterations.
- **Rejection of Monolithic Architecture:** A monolithic approach was rejected as it would require redeployment of the entire system for individual module updates, increasing risk and downtime.

1.4 Scalability and Maintainability Considerations

Scalability:

- Horizontal Scaling: The stateless design of the application layer enables multiple application containers to run behind a load balancer. The system is designed to support up to 10,000 concurrent users (NFR-014) through horizontal scaling.
- Database Scaling: Read replicas may be configured for PostgreSQL to distribute read-heavy queries. Connection pooling via Sequelize is configured with a maximum of 20 connections per container.
- Caching Strategy: Redis is employed to cache frequently accessed data such as product listings, category trees, and search results, ensuring sub-2-second response times (NFR-001).
- CDN Offloading: Static assets are served through AWS CloudFront CDN, reducing latency and server load.

Maintainability:

- Code Organisation: The codebase follows a modular directory structure organised by feature domain, enabling developers to locate and modify code efficiently.
- API Documentation: All public API endpoints are documented using OpenAPI (Swagger) specification, ensuring at least 90% documentation coverage (NFR-017).
- Automated Testing: Unit tests (Jest), integration tests (Supertest), and end-to-end tests are maintained with a target of 90% code coverage.
- Version Control: Git with a branching strategy (main, develop, feature branches) is used for source code management.
- Logging and Monitoring: Structured logging (Winston) and application performance monitoring are implemented across all layers.

CHAPTER 2: DATABASE DESIGN

This chapter presents the database design for the Dickwella Fashion Corner system, including the Entity-Relationship diagram and the normalization rationale.

2.1 Entity-Relationship Diagram

The database is implemented using PostgreSQL 16 and comprises the following primary entities and their relationships, as derived from the data requirements specified in the SRS (Chapter 13).

https://drive.google.com/file/d/1apqRLtY1WECjmlPkBfTGJkgGK_vmZpQF/view?usp=drive_link

Figure 3: Entity-Relationship (ER) Diagram

2.2 Normalization Explanation

The database schema has been designed to conform to the Third Normal Form (3NF) to minimize data redundancy and ensure data integrity.

Table 2: Database Normalization Summary

Normal Form	Requirement	How It Is Satisfied
1NF	All attributes must contain atomic (indivisible) values; each record must be unique.	Each table has a primary key. Multi-valued attributes (e.g., product images, cart items) are stored in separate related tables rather than as comma-separated values.
2NF	All non-key attributes must be fully functionally dependent on the entire primary key.	Composite dependencies are eliminated. For example, ORDER_ITEM stores unit_price separately from PRODUCT_VARIANT.price to preserve the price at the time of purchase.
3NF	No transitive dependencies; non-key attributes must depend only on the primary key.	Transitive dependencies are removed. For example, customer loyalty data is stored in a separate LOYALTY table rather than within the CUSTOMER table.

Additional design decisions include the following. Denormalization for Performance: The CUSTOMER table retains loyalty points and loyalty tier as denormalized fields for display purposes, while the authoritative loyalty data resides in the LOYALTY and LOYALTY_TRANSACTION tables. Soft Deletes: Customer records are not physically deleted upon customer request; instead, a deleted_at timestamp is set, allowing compliance with legal retention obligations. Indexing Strategy: Indexes are created on frequently queried

columns, including CUSTOMER.email, PRODUCT_VARIANT.sku, ORDER.customer_id, ORDER.status, and ORDER.order_date, to ensure database query response times remain below 500ms for standard CRUD operations (NFR-015).

CHAPTER 3: CLASS DIAGRAM

This chapter presents the class diagram for the core domain model of the Dickwella Fashion Corner system, detailing attributes, methods, visibility, relationships, and multiplicity.

3.1 Core Domain Classes

https://drive.google.com/file/d/1yPtr3RkxiE0EcdWsElanHBZ5yxIp3ghf/view?usp=drive_link

Figure 4: Class Diagram – Core Domain Model

3.2 Class Relationships and Multiplicity

Table 3: Class Attributes and Methods Summary

Relationship	Type	Multiplicity	Description
Customer – Address	Composition	1 to 0..*	A customer may have zero or more saved addresses.
Customer – Order	Association	1 to 0..*	A customer may place zero or more orders.
Customer – Cart	Composition	1 to 1	Each customer owns exactly one active cart.
Customer – LoyaltyAccount	Composition	1 to 1	Each registered customer has exactly one loyalty account.
Customer – Review	Association	1 to 0..*	A customer may write zero or more reviews (verified purchases only).
Product – ProductVariant	Composition	1 to 1..*	Each product has at least one variant (size/colour combination).
Product – Review	Association	1 to 0..*	Each product may receive zero or more reviews.
Cart – CartItem	Composition	1 to 0..*	A cart contains zero or more cart items.
Order – OrderItem	Composition	1 to 1..*	An order contains at least one order item.
Order – Return	Association	1 to 0..*	An order may have zero or more associated returns.
Order – PromoCode	Association	0..* to 0..1	An order may optionally use one promotional code.
ChatSession – ChatMessage	Composition	1 to 0..*	A chat session contains zero or more messages.
ProductVariant – InventoryLog	Association	1 to 0..*	Each variant has zero or more inventory change logs.

Visibility notation: Private attributes are denoted with -, and public methods are denoted with +. All domain class attributes are encapsulated as private, with access provided through public getter/setter methods and domain-specific operations.

CHAPTER 4: SEQUENCE DIAGRAMS

This chapter presents sequence diagrams for all major use cases identified in the SRS document. Each diagram illustrates the interactions among system components for a given scenario.

4.1 Customer Registration

https://drive.google.com/file/d/16SUMxUdb0-bKIsRU1x-ObqKLplKGweEi/view?usp=drive_link

Figure 5: Sequence Diagram – Customer Registration

4.2 Product Search and Filtering

https://drive.google.com/file/d/1KaqzAMZin9z_SsUsoP6ib1-XNvCn4BGn/view?usp=drive_link

Figure 6: Sequence Diagram – Product Search and Filtering

4.3 Add to Cart and Checkout

https://drive.google.com/file/d/1fIWyARoXTV9Kj97VXQ_QPgqZXmHdx2qd/view?usp=drive_link

Figure 7: Sequence Diagram – Add to Cart and Checkout

4.4 Order Tracking

https://drive.google.com/file/d/1nnEZiDtactP9Ejr80bFDN3yDStW-s_84/view?usp=drive_link

Figure 8: Sequence Diagram – Order Tracking

4.5 Process Return and Refund

https://drive.google.com/file/d/11mAuZfV36Yzmt-OsxOJTfXPG3aSbPsen/view?usp=drive_link

Figure 9: Sequence Diagram – Process Return and Refund

4.6 Manage Inventory

https://drive.google.com/file/d/1D84_0KLPsGfIbF8YmePHXeAepL3c6bXD/view?usp=drive_link

Figure 10: Sequence Diagram – Manage Inventory

CHAPTER 5: UI/UX DESIGN

This chapter presents the user interface and user experience design for the Dickwella Fashion Corner system, including wireframe descriptions, user flow diagrams, and accessibility considerations.

5.1 Wireframes or Screen Mock-Ups

The following wireframe descriptions outline the key user interface screens. Each wireframe adheres to the design requirements and follows the UI requirements specified in SRS Chapter 12.

Figma Link: https://www.figma.com/design/SKKlLZigL4JafRtfxmRml7/Untitled?node-id=0-1&t=fA3VbJSdV0KhhdHh-1

5.2 User Flow and Navigation Diagrams

https://drive.google.com/file/d/1wFqcPkR3VeRPPpol4ySrSPGQppU42YNF/view?usp=drive_link

Figure 11: User Flow / Navigation Diagram

5.3 Accessibility Considerations

The system is designed to comply with WCAG 2.1 Level AA accessibility guidelines (NFR-008). The following measures are implemented:

Table 4: UI Accessibility Compliance Checklist

WCAG Criterion	Implementation
1.1.1 Non-text Content	All product images include descriptive alt text. Decorative images use empty alt="" attributes.
1.3.1 Info and Relationships	Semantic HTML5 elements are used throughout. Forms use <label> elements associated with inputs.
1.4.3 Contrast (Minimum)	All text meets a minimum contrast ratio of 4.5:1 against its background. Large text meets 3:1.
1.4.4 Resize Text	Text can be resized up to 200% without loss of content or functionality.
2.1.1 Keyboard	All interactive elements are accessible and operable via keyboard.
2.4.1 Bypass Blocks	A "Skip to main content" link is provided at the top of each page.
2.4.7 Focus Visible	A visible focus indicator (outline) is displayed on the currently focused interactive element.
3.1.1 Language of Page	The lang attribute is set on the <html> element (e.g., lang="en").

WCAG Criterion	Implementation
3.3.1 Error Identification	Form validation errors are identified in text and announced to screen readers using ARIA aria-describedby and role="alert".
3.3.2 Labels or Instructions	All form inputs have visible, descriptive labels. Required fields are clearly marked.
4.1.2 Name, Role, Value	Custom UI components use appropriate ARIA roles, states, and properties.

CHAPTER 6: SECURITY DESIGN

This chapter describes the security architecture of the Dickwella Fashion Corner system, covering authentication, authorization, data protection, input validation, and encryption standards.

6.1 Authentication Mechanism

The system employs a JWT (JSON Web Token)-based authentication mechanism for stateless, scalable user session management.

Authentication Flow:

- **Registration:** Customer credentials are collected via the registration form. The password is hashed using the bycrypt algorithm with a minimum work factor of 10 (NFR-012) before being stored in the database. Full card numbers and CVV codes are never stored on the server (NFR-011).
- **Login:** The customer submits email and password. Upon successful authentication, a JWT access token (expiry: 1 hour) and a refresh token (expiry: 7 days, stored in an HTTP-only secure cookie) are issued.
- **Token Structure:** The JWT payload contains the customer ID, email, role (customer/admin), and token expiration timestamp. The token is signed using the HS256 algorithm with a server-side secret key.
- **Token Refresh:** When the access token expires, the client automatically sends the refresh token to obtain a new access token without requiring the user to re-enter credentials.
- **Password Recovery:** A time-limited (1 hour) password reset token is generated and sent to the registered email address. The token is a cryptographically secure random string stored as a hash in the database.

https://drive.google.com/file/d/1MrVrLHg5eIfvYV6bpUDbzIbj7vNg8jls/view?usp=drive_link

Figure 12: Security Architecture Diagram

6.2 Authorization Levels

Role-Based Access Control (RBAC) is implemented to restrict system access based on the authenticated user's role. Three primary roles are defined:

Table 5: Authentication and Authorization Levels

Role	Description	Access Scope
Guest Customer	An unauthenticated visitor browsing the storefront.	Product browsing, search, filtering, product comparison, adding to cart, guest checkout, chatbot interaction.
Registered Customer	An authenticated customer with a verified account.	All guest capabilities plus: wishlist management, order history, order tracking, return requests, reviews and ratings, saved addresses and payment methods, loyalty programme, live chat.
Admin / Inventory Manager	An authenticated administrator with elevated privileges.	Full admin panel access: product and inventory management, order processing, returns and refund processing, customer database, CRM tools, marketing campaigns, analytics and reports.

Middleware Implementation: Every API route is protected by an authorization middleware function that (1) verifies the JWT token, (2) extracts the user role from the token payload, and (3) checks whether the role has permission to access the requested resource. Unauthorized access attempts receive a 403 Forbidden response.

6.3 Data Protection Approach

The following data protection measures are implemented to comply with applicable data protection regulations (e.g., GDPR, local data protection acts) and PCI DSS standards:

- **Data Minimization:** Only data necessary for the system's functionality is collected. Guest checkout requires only email, shipping address, and payment information for the current order.
- **Payment Data Handling:** Full credit card numbers and CVV codes are never transmitted to or stored on the application server. Payment processing is delegated to PCI DSS Level 1 compliant gateways (Stripe, PayPal).
- **Data at Rest:** Sensitive customer data stored in the PostgreSQL database is protected using AES-256 encryption at the storage level via Amazon RDS encryption.

- **Data Retention Policies:** Order data is retained for 7 years, chat logs for 2 years, and inventory logs for 3 years.
- **Data Deletion:** Upon customer account deletion request, personal data is anonymised (soft delete with deleted_at timestamp), while order records are retained for legal compliance.
- **Backup Security:** Database backups (daily full, 6-hourly incremental) are encrypted using AES-256 and stored in a geographically separate AWS S3 bucket.

6.4 Input Validation and Sanitization Strategy

A defence-in-depth approach to input validation is adopted, with validation performed at both the client-side and server-side layers.

Client-Side Validation (React): Real-time form field validation provides immediate user feedback (e.g., email format, password strength meter, required fields). Client-side validation is for user experience only and is never relied upon as a security measure.

Server-Side Validation (Express.js): The express-validator library is used to validate and sanitize all incoming request data before processing. This includes type checking, length constraints, pattern matching, range validation, SQL injection prevention (via Sequelize ORM parameterized queries), XSS prevention (whitelist-based HTML sanitizer), CSRF protection (scurf middleware with SameSite cookie attribute), and rate limiting (100 requests per 15-minute window per IP for general endpoints; 5 login attempts per 15-minute window per IP).

The system is designed to pass OWASP Top 10 vulnerability scans with zero critical or high-severity findings (NFR-013).

6.5 Encryption Standards

Table 6: Encryption Standards Summary

Data Category	Encryption Standard	Implementation
Data in Transit	TLS 1.2+ (NFR-010)	All client-server communication uses HTTPS. HTTP Strict Transport Security (HSTS) headers are set.
Passwords	bcrypt (work factor >= 10)	Passwords are salted and hashed using bcrypt before storage (NFR-012).

Data Category	Encryption Standard	Implementation
Data at Rest (Database)	AES-256	Amazon RDS encryption is enabled for the PostgreSQL database at the storage layer.
Data at Rest (Backups)	AES-256	Backup files on S3 are encrypted using AWS Server-Side Encryption (SSE-S3).
Data at Rest (File Storage)	AES-256	Product images and uploaded files on S3 use server-side encryption.
Payment Data	PCI DSS Tokenisation	Card data is tokenised by the payment gateway (Stripe/PayPal); only tokens are stored.
JWT Tokens	HS256 (HMAC-SHA256)	JWT tokens are signed using a 256-bit secret key to prevent tampering.
Password Reset Tokens	SHA-256	Reset tokens are hashed before database storage; plaintext tokens are sent to user email.
WebSocket Connections	WSS (WebSocket Secure)	All WebSocket connections use the WSS protocol over TLS.

CHAPTER 7: DEPLOYMENT DESIGN

This chapter describes the deployment strategy, hosting plan, hardware requirements, environment specifications, and CI/CD pipeline for the Dickwella Fashion Corner system.

7.1 Hosting Plan

The system is hosted on Amazon Web Services (AWS), selected for its reliability, global availability, comprehensive managed services, and a 12-month free tier suitable for the initial academic development phase. The hosting architecture is designed to support containerized deployment using Docker (NFR-019).

Core AWS Services:

- Amazon EC2 / AWS ECS (Fargate): The Node.js application containers are deployed on AWS ECS using Fargate for serverless container management. During the development phase, a single EC2 t3.medium instance may be used for cost efficiency.
- Amazon RDS (PostgreSQL): A managed PostgreSQL 16 instance serves as the primary database, with automated backups, patching, and optional multi-AZ deployment for high availability.
- Amazon ElastiCache (Redis): A managed Redis 7 cluster provides session storage, query result caching, and rate limiting.
- Amazon S3: Object storage for product images, bulk upload files, generated invoices, and database backups.
- Amazon CloudFront: CDN for delivering static frontend assets with low latency globally.
- AWS Application Load Balancer (ALB): Distributes incoming HTTPS traffic across multiple application containers and handles SSL termination.
- Amazon Route 53: DNS management for the application domain.
- AWS Certificate Manager: Provides free SSL/TLS certificates for HTTPS.

https://drive.google.com/file/d/1DTmz-OdI79x9Mitr9gFw5JfVFOVzFk6v/view?usp=drive_link

Figure 13: Deployment Architecture Diagram

7.2 Hardware Requirements

Table 8: Hardware Requirements

Component	Development / Staging	Production
Application Server	1x EC2 t3.medium (2 vCPU, 4 GB RAM)	2x ECS Fargate tasks (2 vCPU, 4 GB RAM each), auto-scaling to 4
WebSocket Server	Shared with application server	1x ECS Fargate task (1 vCPU, 2 GB RAM), auto-scaling to 2
Database Server	1x RDS db.t3.medium (2 vCPU, 4 GB RAM)	1x RDS db.r6g.large (2 vCPU, 16 GB RAM) + 1 read replica
Cache Server	1x ElastiCache cache.t3.micro (0.5 GB)	1x ElastiCache cache.r6g.large (13 GB)
Object Storage	S3 Standard (estimated 10 GB)	S3 Standard (estimated 100 GB for product images and backups)
CDN	Not required for development	CloudFront distribution (global edge locations)

7.3 Environment Specification

Three environments are maintained to support the development lifecycle:

Table 7: Environment Specification

Attribute	Development	Staging	Production
Purpose	Active development and debugging	Integration testing and UAT	Live system serving end users
URL	localhost:3000 / localhost:5000	staging.dickwellafashion.lk	www.dickwellafashion.lk
Database	Local PostgreSQL or Docker container	RDS instance (separate from production)	RDS instance with read replica
Data	Seed data / mock data	Anonymized copy of production data	Live customer and order data
SSL	Self-signed certificate	AWS Certificate Manager	AWS Certificate Manager
Payment Gateway	Stripe/PayPal test/sandbox mode	Stripe/PayPal test/sandbox mode	Stripe/PayPal live mode
Email Delivery	Console output / Mailtrap sandbox	SendGrid sandbox	SendGrid production
Logging Level	DEBUG	INFO	WARN / ERROR

Attribute	Development	Staging	Production
Access	Development team only	Development team + client (UAT)	Public
Deployment Method	Manual / Docker Compose	Automated via CI/CD pipeline	Automated via CI/CD pipeline

7.4 CI/CD Pipeline Overview

A Continuous Integration / Continuous Deployment pipeline is configured using GitHub Actions to automate testing and deployment:

https://drive.google.com/file/d/1CHuD3RgnFvVAtQ9aU0UR6xFPXXdkLnt0/view?usp=drive_link

Figure 14: CI/CD Pipeline Overview Diagram

Pipeline Stages:

- Build: Dependencies are installed, and the application is compiled.
- Lint: Code quality is enforced using ESLint with a project-specific configuration.
- Test: Unit tests (Jest) and integration tests (Supertest) are executed. A minimum of 90% code coverage is required for the pipeline to pass.
- Security Scan: npm audit is run to detect known vulnerabilities in dependencies.
- Docker Build: A production Docker image is built and tagged with the commit hash.
- Deploy to Staging: The Docker image is deployed to the staging ECS environment for integration testing and UAT.
- Deploy to Production: Upon approval and merge to the main branch, the image is pushed to Amazon ECR and deployed to the production ECS cluster. A health check confirms successful deployment before traffic is routed to the new version.

CHAPTER 8: REFERENCES

[1] IEEE, "IEEE Recommended Practice for Software Requirements Specifications," IEEE Std 830-1998, 1998.

[2] K. E. Wiegers and J. Beatty, Software Requirements, 3rd ed. Redmond, WA, USA: Microsoft Press, 2013.

[3] ISO/IEC, "ISO/IEC 25010:2011, Systems and Software Engineering – Systems and Software Quality Requirements and Evaluation (SQuaRE) – System and Software Quality Models," 2011.

[4] E. Gamma, R. Helm, R. Johnson, and J. Vlissides, Design Patterns: Elements of Reusable Object-Oriented Software. Boston, MA, USA: Addison-Wesley, 1994.

[5] C. Richardson, Microservices Patterns: With Examples in Java. Shelter Island, NY, USA: Manning Publications, 2018.

[6] Stripe, Inc., "Stripe API Reference," [Online]. Available: https://stripe.com/docs/api. [Accessed: Mar. 31, 2026].

[7] PayPal, Inc., "PayPal Developer Documentation," [Online]. Available: https://developer.paypal.com/docs/. [Accessed: Mar. 31, 2026].

[8] Twilio SendGrid, "SendGrid Email API Documentation," [Online]. Available: https://docs.sendgrid.com/. [Accessed: Mar. 31, 2026].

[9] W3C, "Web Content Accessibility Guidelines (WCAG) 2.1," [Online]. Available: https://www.w3.org/TR/WCAG21/. [Accessed: Mar. 31, 2026].

[10] PCI Security Standards Council, "PCI Data Security Standard (PCI DSS)," [Online]. Available: https://www.pcisecuritystandards.org/. [Accessed: Mar. 31, 2026].

[11] IETF, "RFC 6455 – The WebSocket Protocol," [Online]. Available: https://tools.ietf.org/html/rfc6455. [Accessed: Mar. 31, 2026].

[12] IETF, "RFC 8030 – Generic Event Delivery Using HTTP Push," [Online]. Available: https://tools.ietf.org/html/rfc8030. [Accessed: Mar. 31, 2026].

[13] Meta Platforms, Inc., "React – A JavaScript library for building user interfaces," [Online]. Available: https://react.dev/. [Accessed: Mar. 31, 2026].

[14] OpenJS Foundation, "Node.js Documentation," [Online]. Available: https://nodejs.org/docs/. [Accessed: Mar. 31, 2026].

[15] OpenJS Foundation, "Express.js – Fast, unopinionated, minimalist web framework for Node.js," [Online]. Available: https://expressjs.com/. [Accessed: Mar. 31, 2026].

[16] The PostgreSQL Global Development Group, "PostgreSQL Documentation," [Online]. Available: https://www.postgresql.org/docs/. [Accessed: Mar. 31, 2026].

[17] Sequelize Authors, "Sequelize ORM Documentation," [Online]. Available: https://sequelize.org/. [Accessed: Mar. 31, 2026].

[18] Redis Ltd., "Redis Documentation," [Online]. Available: https://redis.io/docs/. [Accessed: Mar. 31, 2026].

[19] Docker, Inc., "Docker Documentation," [Online]. Available: https://docs.docker.com/. [Accessed: Mar. 31, 2026].

[20] Amazon Web Services, Inc., "AWS Documentation," [Online]. Available: https://docs.aws.amazon.com/. [Accessed: Mar. 31, 2026].

[21] OWASP Foundation, "OWASP Top Ten," [Online]. Available: https://owasp.org/www-project-top-ten/. [Accessed: Mar. 31, 2026].

[22] GitHub, Inc., "GitHub Actions Documentation," [Online]. Available: https://docs.github.com/en/actions. [Accessed: Mar. 31, 2026].